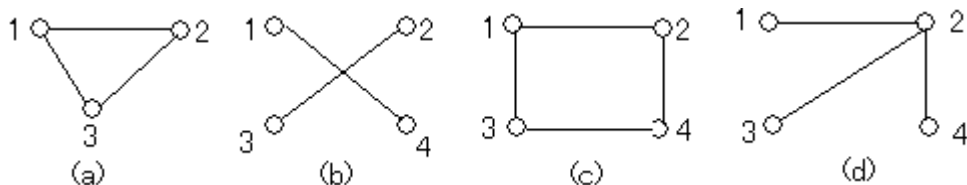
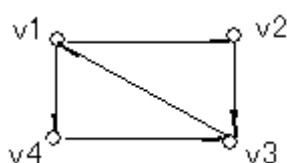


1 在 (下图) 所示的各无向图中:



- (1) 找出所有的简单环。
- (2) 哪些图是连通图? 对非连通图给出其连通分量。
- (3) 哪些图是自由树 (或森林)?

2 在下图所示的有向图中:



- (1) 该图是强连通的吗? 若不是, 则给出其强连通分量。
- (2) 请给出所有的简单路径及有向环。
- (3) 请给出每个顶点的度, 入度和出度。
- (4) 请给出其邻接表、邻接矩阵及逆邻接表。

3 假设图的顶点是 A, B, ..., 请根据下述的邻接矩阵画出相应的无向图或有向图。

	A	B	C	D	E	F	G
A	0	1	1	1	0	0	0
B	1	0	1	1	0	0	0
C	1	1	0	1	0	0	0
D	1	1	1	0	1	0	0
E	0	0	0	1	0	1	0
F	0	0	0	0	1	0	1
G	0	0	0	0	0	1	0

4 假设一棵完全二叉树包括 A, B, C, ... 等七个结点, 写出其邻接表和邻接矩阵。

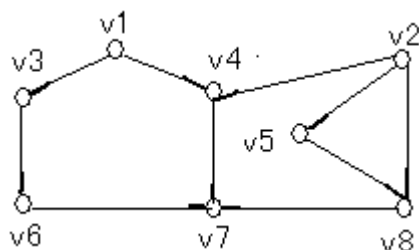
5 对 n 个顶点的无向图和有向图, 采用邻接矩阵和邻接表表示时, 如何判别下列有关问题?

- (1) 图中有多少条边?
- (2) 任意两个顶点 i 和 j 是否有边相连?
- (3) 任意一个顶点的度是多少?

6 n 个顶点的连通图至少有几条边? 强连通图呢?

7 DFS 和 BFS 遍历各采用什么样的数据结构来暂存顶点? 当要求连通图的生成树的高度最小, 应采用何种遍历?

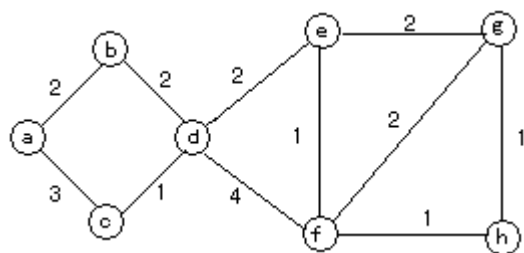
8 画出以顶点 v_1 为初始源点遍历图 25(下图)所示的有向图所得到的 DFS 和 BFS 生成森林。



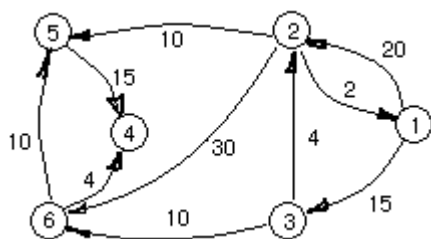
9 按顺序输入顶点对: $(1, 2), (1, 6), (2, 6), (1, 4), (6, 4), (1, 3), (3, 4), (6, 5), (4, 5), (1, 5), (3, 5)$, 根据第 2.2 节中算法 CreatALGraph 画出相应的邻接表。并写出在该邻接表上, 从顶点 4 开始搜索所得的 DFS 和 BFS 序列, 及 DFS 和 BFS 生成树。

10 什么样的图其最小生成树是唯一的? 用 PRIM 和 Kruskal 求最小生成树的时间各为多少? 它们分别适合于哪类图?

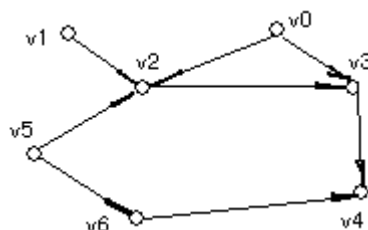
11 对下图所示的连通图, 请分别用 Prim 和 Kruskal 算法构造其最小生成树。



12 对下图所示的有向图, 试利用 Dijkstra 算法求出从源点 1 到其它各顶点的最短路径, 并写出执行算法过程中扩充红点集的每次循环状态(见表 7.2)。



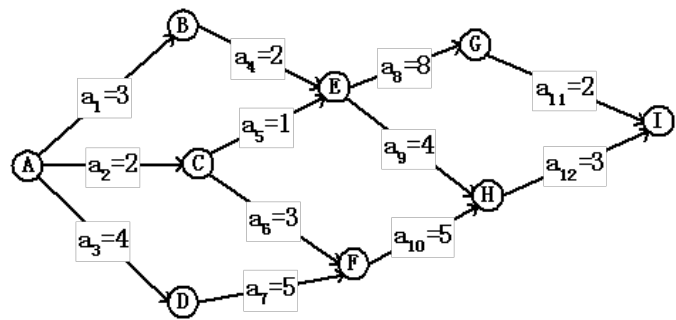
13 试写出下图所示有向图的所有拓扑序列, 并指出就用 7.6 节给出的 NonPreFirstTopSort 算法求得的是哪个序列, 设邻接表的边表结点中的邻接点序号是递增有序的。



14 什么样的 DAG 的拓扑序列是唯一的？

15 请以 V0 为源点，给出用 DFS 搜索图 7.28（上图）得到的逆拓扑序列。

16 对于下面的 AOE 网，请求出一条关键路径（列出所有顶点事件的最早发生时间 VE、最晚发生时间 VL，和所有活动的最早发生时间 E、最迟发生时间 L）。



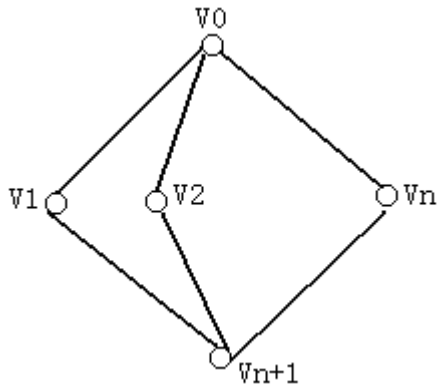
顶点	VE	VL	活 动	E	L
A			a1		
B			a2		
C			a3		
D			a4		
E			a5		
F			a6		
G			a7		
H			a8		
I			a9		
			a10		
			a11		
			a12		

第七章 图（算法设计）

17 试在无向图的邻接矩阵和邻接链表上实现如下算法：

- (1) 往图中插入一个顶点
- (2) 往图中插入一条边
- (3) 删去图中某顶点
- (4) 删去图中某条边

18 下面的伪代码是一个广度优先搜索算法，试以图 7.29(下图)中的 v_0 为源点执行该算法，请回答下述问题：



- (1) 对图中顶点 v_{n+1} ，它需入队多少次？它被重复访问多少次？
- (2) 若要避免重复访问同一个顶点的错误，应如何修改此算法？

```
void BFS(ALGraph *G, int k)
{
    //以下省略局部变量的说明，visited 各分量初值为假
    InitQueue(&Q); //置空队列
    EnQueue(&Q, k); //k 入队
    while (!QueueEmpty(&Q)) {
        i = DeQueue(&Q); //vi 出队
        visited[i] = TRUE; //置访问标记
        printf("%c", G->adjlist[i].vertex); //访问 vi
        for (p = G->adjlist[i].firstedge; p; p = p->next)
            //依次搜索 vi 的邻接点 vj (不妨设 p->adjvex=j)
            if (!visited[p->adjvex]) //若 vj 没有访问过
                EnQueue(&Q, p->adjvex); //vj 入队
    } //endofwhile
} //BFS
```

19 试以邻接表和邻接矩阵为存储结构，分别写出基于 DFS 和 BFS 遍历的算法来判别顶点 v_i 和 v_j ($i < j$) 之间是否有路径。

20 试分别写出求 DFS 和 BFS 生成树(或生成森林)的算法，要求打印出所有的树边。

21 写一算法求连通分量的个数并输出各连通分量的顶点集。

22 设图中各边的权值都相等，试以邻接矩阵和邻接表为存储结构，分别写出算法：

- (1) 求顶点 v_i 到顶点 v_j ($i < j$) 的最短路径
- (2) 求源点 v_i 到其余各顶点的最短路径

要求输出路径上的所有顶点(提示：利用 BFS 遍历的思想)

23 以邻接表为存储结构，写一个基于 DFS 遍历策略的算法，求图中通过某顶点 v_k 的简单回路(若存在)。

24 写一算法求有向图的所有根(若存在), 分析算法的时间复杂度。

25 改写 7.5 节的算法 Print, 使输出的从源点到各终点的最短路径是正向的。(提示: 使用栈暂存路径)

26 对 7.6 节的 NonSuccFirstTopSort 算法求精, 分别以邻接矩阵和邻接表作为存储结构, 写出其具体算法, 并分析算法的时间。

27 设一个有向图 DAG, 试以邻接矩阵和邻接表作为存储结构, 写出对 7.6 节的 DFSTopSort 求精算法。为什么有向图不是 DAG 时, 该算法不能正常工作?

28 利用拓扑排序算法的思想写一算法判别有向图中是否存在有向环, 当有向环存在时, 输出构成环的顶点。